

Physics-Informed Neural Operator-Based Poisson Solver

Dhruvil Patel
ID: 202301201

Om Patel
ID: 202301163

Abstract—Every time-step of a Particle-in-Cell (PIC) plasma simulation requires a new solution of the 2-D electrostatic Poisson equation. Classical sparse-matrix solvers provide accurate fields, but their cost grows unfavorably with grid resolution and often makes the field solve a dominant runtime bottleneck. This work replaces that repeated algebraic solve with a Deep Operator Network (DeepONet) that learns the continuous operator mapping charge density to electrostatic potential. Unlike a convolutional surrogate tied to a fixed grid, the proposed architecture couples a Fourier Neural Operator branch, which captures global non-local field dependencies in the frequency domain, with a coordinate-based trunk network that evaluates the solution at arbitrary spatial locations. A composite physics-informed loss, combining supervised reconstruction, a finite-difference PDE residual, and electric-field gradient alignment, moves training beyond curve fitting toward Poisson-consistent prediction. Exact Dirichlet boundary conditions are enforced by hard boundary clamping after each forward pass, avoiding the compromise of soft penalty terms. Evaluated on charge distributions drawn from seven plasma scenarios, the surrogate preserves localized density spikes while maintaining sub-4% absolute percentage error and delivering a substantial wall-clock speedup over a direct CPU solver.

Index Terms—Poisson equation, DeepONet, Fourier Neural Operator, Particle-in-Cell, plasma simulations, electrostatics.

TABLE I
NOTATION REFERENCE.

Symbol	Meaning	Units
Ω	2-D computational spatial domain	—
$\partial\Omega$	Grounded domain boundaries	—
$\mathcal{G}$	Nonlinear operator mapping $\rho \mapsto \phi$	—
$\phi(x, y)$	Electrostatic potential	V
$\rho(x, y)$	Charge density	C/m ³
$\mathbf{E}$	Electric field ($-\nabla\phi$)	V/m
ϵ_0	Permittivity of free space	F/m
∇^2	Laplacian operator	m ⁻²
$\tilde{\phi}, \tilde{\rho}$	Characteristic normalised fields	—
ρ_{norm}	Charge normalisation constant	C/m ³
ϕ_{norm}	Potential normalisation constant	V
h	Uniform spatial grid spacing	m
K	Number of basis channels	4
$b_k(x, y)$	Branch network spatial field	—
$t_k(x, y)$	Trunk network coordinate weight	—
$\mathcal{L}_{\text{data}}$	Data reconstruction loss (MSE)	—
$\mathcal{L}_{\text{pde}}$	Physics-informed PDE residual loss	—
$\mathcal{L}_{\text{grad}}$	Spatial gradient alignment loss	—
λ_{eff}	Dynamic physics loss weighting	—
N	Grid resolution points per axis	256

I. INTRODUCTION

Plasma simulations underpin research in fusion energy, microwave–plasma interaction, and astrophysical modelling [3], [7]. The electrostatic Particle-in-Cell (PIC) method is a central tool for kinetic modelling of low-temperature plasma (LTP), because it resolves particle motion while updating the self-consistent electric field. Its inner loop repeats three operations at every time-step: (i) deposit macro-particle charges onto a spatial grid, (ii) solve the resulting field equation, and (iii) interpolate the fields back to push particles. Stage (ii) solves the 2-D Poisson equation

$$\nabla^2\phi(x, y) = -\frac{\rho(x, y)}{\epsilon_0}, \quad (1)$$

where ϕ is the electrostatic potential produced by the charge distribution ρ , and ϵ_0 is the vacuum permittivity. In practice, this field solve consumes over 30% of total PIC runtime, so reducing its cost directly improves the feasibility of long plasma simulations.

Why Classical Solvers Are Not Enough

Direct sparse solvers such as LU factorisation provide reliable reference solutions, but they become expensive on large $N \times N$ grids. More importantly, the charge density ρ changes at every time-step, so the field solve must be repeated even though the governing operator and boundary conditions remain fixed. Iterative methods such as Conjugate Gradient reduce the per-step cost in some regimes, but they still require multiple iterations and can have input-dependent runtime. The central opportunity is therefore clear: the Poisson operator is unchanged while only its input varies, suggesting that a learned surrogate can amortise the cost of solving across many PIC time-steps.

Why Standard Neural Surrogates Also Fail

A natural first surrogate is a convolutional U-Net [3] that maps ρ directly to ϕ . This approach is useful but limited for the Poisson problem. Convolutional layers aggregate information locally, whereas electrostatic potential is globally coupled: charge at one location influences the field throughout the domain. Capturing this dependence with local filters requires substantial depth and remains tied to the discretisation on which the filters were trained. If the grid resolution changes, the learned pixel-space relationships no longer represent the same physical distances and the model must be retrained.

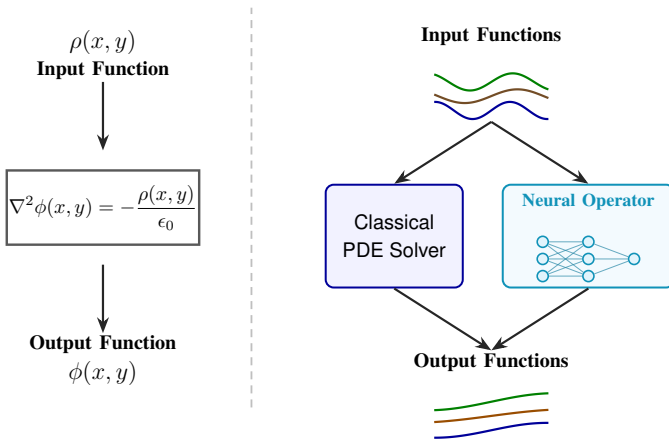


Fig. 1. Operator learning paradigm for electrostatic systems. **Left:** Mathematical abstraction—the operator $\mathcal{G}$ maps the entire source charge distribution $\rho(x, y)$ to the continuous potential field $\phi(x, y)$. **Right:** Computational realization—classical PDE solvers (blue) and neural operators (cyan) both map input functions to output functions, but the neural operator learns the underlying continuous mapping $\mathcal{G}$ from data and evaluates it in a single $\mathcal{O}(1)$ forward pass.

Physics-informed neural networks (PINNs) [4] offer a principled alternative by embedding the PDE residual directly into the training loss. However, a standard PINN learns the solution surface for one particular charge distribution ρ . When ρ changes—which in a live PIC loop occurs at every time-step—the PINN must be retrained or reformulated. For repeated field solves, the single-instance formulation therefore does not address the operational bottleneck.

Operator Learning as the Logical Resolution

Neural operators address these limitations by learning the mapping itself rather than a single solution. Instead of memorising one field, they approximate the continuous operator $\mathcal{G} : \rho \mapsto \phi$ that maps a charge distribution to its corresponding potential. DeepONet [5] and the Fourier Neural Operator (FNO) [6] are leading architectures in this class, with theoretical foundations for approximating operators in infinite-dimensional function spaces. This work combines their complementary strengths in a multi-scale branch–trunk architecture: an FNO branch encodes the global spatial structure of ρ in the frequency domain, while a coordinate-based trunk network evaluates the learned solution at any point in the domain. The resulting forward pass replaces repeated matrix factorisation with a fixed sequence of FFT operations and dense projections.

Contributions

- A DeepONet surrogate that replaces the 2-D electrostatic Poisson solve inside a PIC loop, maintaining 3.5599% average absolute percentage error across 4002 unseen charge configurations drawn from seven distinct plasma scenarios.
- Analysis of the architecture’s mesh-invariant formulation, which combines the Fourier Neural Operator’s spectral weight parameterisation with coordinate-based trunk

queries to support zero-shot evaluation on grids differing from the training resolution.

- A multi-objective composite loss incorporating data-driven Mean Squared Error (MSE), a normalized finite-difference PDE residual, and a spatial gradient-alignment term, so the surrogate minimizes pointwise potential error while preserving the physical consistency of the derived electric field.

II. METHODOLOGY

A. Data Generation

All training and validation data originate from a C++ SYCL-accelerated PIC code that models plasma evolution self-consistently. This choice is deliberate: a solver intended for live inference must learn from the same statistical distribution of charge fields it will encounter inside the PIC loop, not from simplified analytical profiles. The generation pipeline proceeds in four stages:

- 1) **Charge deposition.** At selected timesteps the macro-particle coordinates are mapped onto a 256×256 mesh via bilinear weighting, producing $\rho_{i,j}$.
- 2) **Matrix assembly.** The 2-D finite-difference Laplacian $\mathbf{A}$ is built with grounded Dirichlet conditions ($\phi = 0$ on every boundary node).
- 3) **Direct solve.** Intel MKL Pardiso solves $\mathbf{A}\vec{\phi} = -\vec{\rho}/\epsilon_0$, giving the reference potential $\phi_{i,j}$.
- 4) **Serialisation.** Each (ρ_k, ϕ_k) pair is written to HDF5 so that C++–Python data transfer incurs zero precision loss.

The HDF5 serialisation step is important for reproducibility. Floating-point data exchanged through text files or untyped binary streams can accumulate rounding errors; a typed, chunk-compressed HDF5 format ensures that the Python training loop ingests bit-identical copies of the C++ solver outputs. We reserved 4,002 snapshots for validation. To prevent the network from memorising a single charge topology, the PIC solver was run under seven distinct plasma scenarios (Table II), spanning electric-field-only discharges through high-pressure collisional regimes with and without applied magnetic fields.

III. DATA PRE-CONDITIONING AND NORMALIZATION

A. Scale Discrepancy and Normalisation

Before a neural network can process raw PIC data effectively, a basic numerical obstacle must be resolved. The charge density ρ and electrostatic potential ϕ occupy very different physical scales: raw PIC snapshots span roughly eight orders of magnitude, with $\rho \sim \mathcal{O}(10^{-8})$ C/m³ versus $\phi \sim \mathcal{O}(1)$ V. If these quantities are used without rescaling, the larger-magnitude signal dominates the optimization, while gradients associated with the smaller-scale field become numerically weak. Training then becomes unstable or stalls.

Plasma charge fields are also highly uneven: a narrow spike can be much larger than the background. Min–max scaling would let such spikes determine the scale and compress most cells into a narrow range near zero, obscuring the variation that the network must learn. We therefore normalise by the

TABLE II
C++ PIC SIMULATION SCENARIOS USED FOR DATASET GENERATION.

Simulation Case	Physical Characteristics	Significance for Training
Case 1	Electric field only	Captures pure electrostatic electron–ion interaction.
Case 2	High magnetic field	Uniform B -field; introduces $E \times B$ drift physics.
Case 3	Collision, low pressure	Collisional transport in the low-pressure regime.
Case 4	Collision, high pressure	Collisional transport in the high-pressure regime.
Case 5	Ionisation, high pressure	Ionisation-driven transport at high pressure.
Case 6	Collision + B -field, low pressure	Collisional transport with applied B at low pressure.
Case 7	Collision + B -field, high pressure	Collisional transport with applied B at high pressure.

global standard deviation of the training and validation data. The standard deviation provides a characteristic scale for each field, bringing both variables to comparable numerical ranges without erasing physically meaningful localized spikes.

$$\tilde{\rho}_{i,j} = \frac{\rho_{i,j}}{\rho_{\text{norm}}}, \quad \tilde{\phi}_{i,j} = \frac{\phi_{i,j}}{\phi_{\text{norm}}} \quad (2)$$

with $\rho_{\text{norm}} = 1.23223 \times 10^{-8} \text{ C/m}^3$ and $\phi_{\text{norm}} = 9.21588 \times 10^{-1} \text{ V}$, both computed once from the data and held constant across training and inference. Both fields now sit at $\mathcal{O}(1)$, restoring healthy gradient flow while letting isolated charge spikes persist as large—but not catastrophically large—values. Cases 1–4 were used for training; Case 5 was used for validation; Cases 6–7 are used for testing.

B. Preservation of the Poisson Structure

Substituting $\phi = \phi_{\text{norm}} \tilde{\phi}$ and $\rho = \rho_{\text{norm}} \tilde{\rho}$ into Eq. (1) and dividing through by ϕ_{norm} yields

$$\nabla^2 \tilde{\phi} = -C \tilde{\rho}, \quad C = \frac{\rho_{\text{norm}}}{\epsilon \phi_{\text{norm}}} \approx 1510. \quad (3)$$

This normalization preserves the Poisson structure without distorting the underlying physics. However, the coefficient

$C \approx 1510$ can dominate the loss landscape if the residual is used without weighting. We therefore introduce a balancing weight λ on the physics-informed loss term so that the PDE residual and data loss contribute gradients of comparable scale during training.

C. Boundary Preservation

The deliberate omission of mean subtraction has a concrete physical consequence: grounded walls, where the physical boundary condition requires $\phi = 0$, remain exactly zero after rescaling,

$$\tilde{\phi}(x_{\partial\Omega}) = \frac{0}{\phi_{\text{norm}}} = 0. \quad (4)$$

Subtracting the training-set mean would shift these boundary cells to small non-zero floating-point values. That apparently minor numerical artifact would inject spurious boundary gradients into the PIC particle pusher at every time-step and accumulate as drift in long-time plasma dynamics.

IV. NEURAL ARCHITECTURE

A. Operator-Learning Formulation

The core abstraction of this work is a shift in what the neural network is asked to learn. A standard regression network learns a fixed function $\phi(x, y)$ for one particular charge distribution ρ . Here, the network learns the *operator* $\mathcal{G}$ that maps any admissible charge field ρ to its corresponding potential field ϕ :

$$\mathcal{G} : V \rightarrow C(K), \quad \phi = \mathcal{G}(\rho). \quad (5)$$

This distinction is central to the proposed solver. Once trained, an operator-learned network can be evaluated on new charge distributions without retraining. The theoretical justification for approximating such operators with finite networks is the Chen–Chen universal approximation theorem [2]: for any continuous operator $\mathcal{G}$ and any tolerance $\varepsilon > 0$, there exist finite networks b_k and t_k such that

$$\max_{(u,y) \in U \times \Omega} \left| \mathcal{G}(\rho)(y) - \sum_{k=1}^K b_k(\rho(x_1), \dots, \rho(x_m)) t_k(y) \right| < \varepsilon. \quad (6)$$

This decomposition provides the blueprint for a two-sub-network architecture [5]: one network encodes the input function, another encodes the query coordinate, and their outputs are combined to reconstruct the solution field.

B. Target Variable Selection

We predict the scalar potential ϕ rather than the electric field $\vec{E}$ directly. The physical reason is straightforward: grounded walls impose $\phi = 0$, a fixed anchor that the network can be explicitly forced to satisfy. The electric field at the wall, by contrast, depends on the interior plasma state and changes with each charge distribution. Predicting ϕ and recovering $\vec{E}$ by finite differences is therefore the formulation that naturally admits a hard boundary constraint. Direct prediction of $\vec{E}$ would require boundary enforcement on a vector field with no fixed reference value.

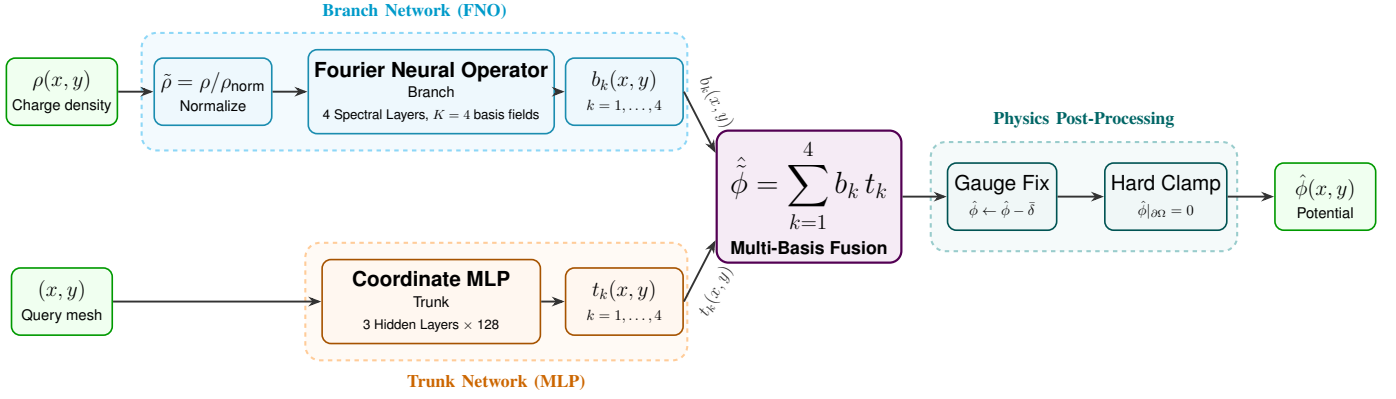


Fig. 2. The Multi-Basis DeepONet architecture for the 2-D Poisson solver. The **Branch** (cyan) encodes the normalized charge density $\tilde{\rho}$ through four spectral Fourier layers into $K = 4$ spatial basis fields b_k . The **Trunk** (orange) maps continuous query coordinates to $K = 4$ geometric weights t_k . **Multi-basis fusion** (violet) reconstructs the normalized potential $\hat{\phi}$, which is gauge-corrected and hard-clamped (teal) to enforce exact Dirichlet boundaries before output.

C. Branch Network: Fourier Neural Operator

The central challenge in encoding a charge density field is the non-locality of the Poisson equation: the potential at any point is influenced by charge throughout the domain, not only in an immediate neighbourhood. A conventional convolutional encoder, whose receptive field is determined by kernel size and depth, cannot capture these long-range dependencies efficiently.

We therefore use a Fourier Neural Operator (FNO) [6]. The FNO transforms the full spatial field into the frequency domain via a 2-D FFT, applies learned linear weights in spectral space, and transforms the result back. In the frequency domain, every output mode can depend on all input modes; a single spectral layer has a global receptive field by construction. This aligns naturally with the non-local structure of the Poisson operator.

The branch network employs a two-dimensional FNO with one input channel (normalised charge density $\tilde{\rho}$) and $p = 4$ output channels. The internal architecture consists of:

- 1) A lifting layer mapping the scalar input $\tilde{\rho} \in \mathbb{R}^{1 \times H \times W}$ to a latent feature space of dimension $c = 64$.
- 2) Four stacked Fourier layers, each computing the forward 2-D FFT, applying a learned complex-valued linear transformation $R_\phi \in \mathbb{C}^{c \times c \times k_{\max}}$ to the lowest $k_{\max} = 24$ modes per spatial dimension, and inverting the transform.
- 3) A two-layer decoder of width 64 projecting the latent features to $p = 4$ spatial fields $\{b_k(\rho; x, y)\}_{k=1}^4$.

Padding of 12 grid points is applied in physical space to accommodate the spectral bandwidth of 24 modes. The truncation of modes above $k_{\max}$ is deliberate: it acts as a low-pass filter that suppresses high-frequency noise while retaining the physically meaningful spatial structures [6].

D. Trunk Network: Coordinate MLP

The branch network encodes the charge distribution, while the trunk network encodes spatial location. Specifically, the trunk is a fully connected multilayer perceptron (MLP) that maps a continuous coordinate pair $(x, y) \in [0, 1]^2$ to a four-dimensional basis vector, configured with 3 hidden layers of

width 128 and SiLU activations. The coordinate grids are flattened to shape $(B, N, 2)$ with $N = H \times W$, yielding output $(B, N, 4)$ reshaped to $(B, 4, H, W)$.

Because the trunk receives floating-point coordinates rather than discrete pixel indices, the same trained weights can be queried at any spatial density without modification. This property is essential for deployment at resolutions other than the training grid (Sec. IV-G).

E. Multi-Basis Fusion

The Chen–Chen theorem (Eq. (6)) decomposes the target operator into a sum of products between input-dependent and coordinate-dependent terms. The multi-basis fusion layer implements this decomposition directly. The FNO branch produces $p = 4$ spatial fields $b_k(\rho; x, y)$, each capturing a different spectral mode of the charge structure. The trunk produces $p = 4$ coordinate basis fields $t_k(x, y)$, each representing a learned geometric weight. The predicted normalised potential is their element-wise dot product summed over channels:

$$\hat{\phi}(x, y; \rho) = \sum_{k=1}^4 \underbrace{b_k(\rho; x, y)}_{\text{FNO branch}} \cdot \underbrace{t_k(x, y)}_{\text{MLP trunk}}. \quad (7)$$

The four basis channels give the network enough expressivity to represent the qualitatively different potential topologies that arise from diffuse backgrounds, localised plasma sheaths, and multi-peak charge configurations.

F. Hard Boundary Enforcement

In classical PINNs, boundary conditions are often enforced by adding a penalty term to the loss function. Such penalties push predictions toward zero at the walls but do not enforce exact satisfaction; residual boundary errors can remain even after convergence. In a PIC context these errors are not cosmetic: the particle pusher interpolates $\vec{E} = -\nabla\phi$ to update particle trajectories, and a non-zero wall gradient injects a spurious electric field at every time-step. Over thousands of time-steps, this can accumulate into unphysical energy gain.

Structure of a Single Fourier Neural Operator (FNO) Layer

Spectral path captures global structure; local path preserves fine spatial detail

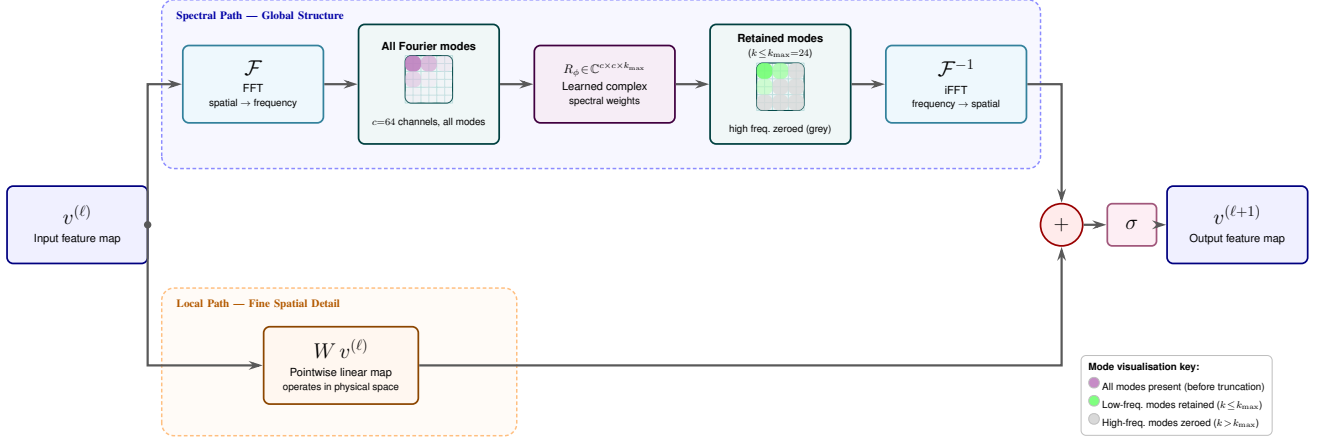


Fig. 3. Internal structure of a single Fourier Neural Operator (FNO) layer, the core building block of the branch network. The input feature map $v^{(\ell)}$ is split into two parallel paths. **Spectral path** (top, blue region): *Step 1* — An FFT decomposes the input into frequency-domain coefficients across all $c = 64$ latent channels. In the left grid visualisation, darker purple cells represent stronger low-frequency components; lighter cells represent weaker high-frequency components that are still present. *Step 2* — Learned complex weights $R_\phi \in \mathbb{C}^{c \times c \times k_{\max}}$ ($c = 64$, $k_{\max} = 24$ modes per spatial dimension) scale and mix the Fourier coefficients; this is the *only* trainable operation in the spectral path. *Step 3* — Modes above $k_{\max}$ are zeroed (grey cells in the right grid), acting as a low-pass filter. Green cells show the retained low-frequency modes encoding smooth, large-scale spatial structure. *Step 4* — An inverse FFT converts the filtered result back to physical space. **Local path** (bottom, orange region): A pointwise linear map W acts directly in physical space, preserving sharp gradients and fine-scale features that frequency-domain truncation would remove. Both outputs are summed element-wise and passed through activation σ to yield $v^{(\ell+1)}$. Because R_ϕ operates on a fixed number of modes independent of grid size N , the same trained layer evaluates at any resolution without retraining (Sec. IV-G).

TABLE III
DEEPONET ARCHITECTURE CONFIGURATION.

Component	Parameter	Value
Multi-basis channels	p	4
FNO latent channels	c	64
FNO spectral modes	$k_{\max}$	24
FNO layers	—	4
FNO padding	—	12
FNO decoder width	—	64
Trunk hidden layers	—	3
Trunk width	—	128
Grid resolution	N	256
Total parameters	—	$\approx 1.6 \times 10^6$

We avoid this entirely by modifying the output after the forward pass. A gauge correction first removes the mean predicted boundary value to select the grounded Dirichlet gauge, and then all boundary edges are explicitly zeroed:

$$\hat{\phi}_{\text{corrected}} = \hat{\phi}_{\text{raw}} - \delta^{(n)}, \quad \hat{\phi}|_{\partial\Omega} = 0. \quad (8)$$

Because $\delta^{(n)}$ is a spatially constant offset, $\nabla^2 \delta^{(n)} = 0$; the correction shifts the global field without altering the interior physics. Boundary compliance is therefore exact by construction and requires no penalty tuning.

G. Theoretical Mesh Invariance and Resolution Independence

Convolutional PDE surrogates are discretisation-locked: a model trained on a 256×256 grid encodes spatial relationships

in pixel space, and those relationships lose physical meaning at any other resolution. Our architecture does not share this limitation.

Both sub-networks operate on continuous mathematical objects rather than fixed pixel indices:

- 1) **Spectral independence (Branch).** The FNO parameterises the integral operator kernel in the continuous Fourier domain. Each Fourier layer computes

$$(\mathcal{K}u)(x) = \mathcal{F}^{-1}(R_\phi \cdot (\mathcal{F}u))(x), \quad (9)$$

where $R_\phi \in \mathbb{C}^{c \times c \times k_{\max}}$ holds a fixed number of learned complex weights, independent of the grid resolution N . The DFT/iDFT pair can be evaluated at any N using the same R_ϕ .

- 2) **Coordinate independence (Trunk).** The trunk evaluates continuous floating-point coordinates $(x, y) \in [0, 1]^2$. Querying at 128, 256, or 1024 uniformly spaced points reuses the same weights unchanged.

A model trained at 256×256 is therefore theoretically capable of zero-shot evaluation at 512×512 or finer. This confirms that the learned object is the continuous operator $\mathcal{G} : \rho \mapsto \phi$, not a discrete pixel-to-pixel regression. Empirical validation at resolutions beyond 256×256 is left for future work.

V. PHYSICS-INFORMED TRAINING

A. Composite Loss Function

A data-only loss teaches the model to match the training targets, but it does not ensure that predictions satisfy the Poisson equation on new charge maps. A physics-only loss enforces

the equation, but it may drift away from the numerical values produced by the reference PIC solver. A gradient-alignment loss keeps the spatial slope of the predicted potential close to the reference, helping the derived electric field remain stable and physically meaningful. We therefore train with all three losses together.

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{data}} + \lambda_{\text{eff}}(e) \mathcal{L}_{\text{pde}} + \lambda_{\nabla} \mathcal{L}_{\text{grad}}, \quad (10)$$

with $\lambda_{\text{eff}}(e)$ ramped linearly over early epochs (Sec. V) and $\lambda_{\nabla} = 0.05$ fixed.

1) *Data reconstruction loss*: The data term penalises pointwise deviation from the Pardiso reference solution:

$$\mathcal{L}_{\text{data}} = \frac{1}{HW} \sum_{i,j} \left(\hat{\phi}_{i,j} - \tilde{\phi}_{i,j}^{\text{true}} \right)^2. \quad (11)$$

This is the main supervised signal, pushing the prediction to match the Pardiso reference at each grid point. Used alone, however, it can produce smooth fields that fit the data while violating the Poisson equation in parts of the domain.

2) *PDE residual loss*: To ensure that the prediction also follows the Poisson equation, we evaluate the residual directly from the predicted field. At each grid cell, the residual is

$$r_{i,j} = -\nabla_h^2 \hat{\phi}_{i,j} - C \tilde{\rho}_{i,j}, \quad (12)$$

where the Laplacian is approximated by the standard 5-point centred finite-difference stencil:

$$\nabla_h^2 f_{i,j} = \frac{f_{i+1,j} + f_{i-1,j} + f_{i,j+1} + f_{i,j-1} - 4f_{i,j}}{h^2}. \quad (13)$$

The raw residual grows with the source magnitude. If left unscaled, high-charge samples would dominate training. We therefore normalise the residual so each sample is penalised by relative physics error, not only by absolute magnitude:

$$\mathcal{L}_{\text{pde}} = \frac{\frac{1}{|\Omega_{\text{int}}|} \sum_{(i,j) \in \Omega_{\text{int}}} |r_{i,j}|}{\text{mean}[C\tilde{\rho}] + 10^{-12}}. \quad (14)$$

Only interior points Ω_{int} are used. We crop 2 cells from each boundary so edge errors from finite differences do not affect the physics signal.

3) *Electric-field gradient loss*: In PIC, particle motion is updated using $\vec{E} = -\nabla\phi$. Small spatial noise in ϕ can therefore create artificial force noise even when pointwise voltage error is low. We address this by matching the predicted and reference gradients in both spatial directions:

$$\mathcal{L}_{\text{grad}} = \text{MSE}\left(\partial_x \hat{\phi}, \partial_x \tilde{\phi}^{\text{true}}\right) + \text{MSE}\left(\partial_y \hat{\phi}, \partial_y \tilde{\phi}^{\text{true}}\right), \quad (15)$$

Derivatives are computed with centred finite differences for both predicted and reference fields. This keeps the loss consistent with the PDE residual term and improves electric-field smoothness during inference.

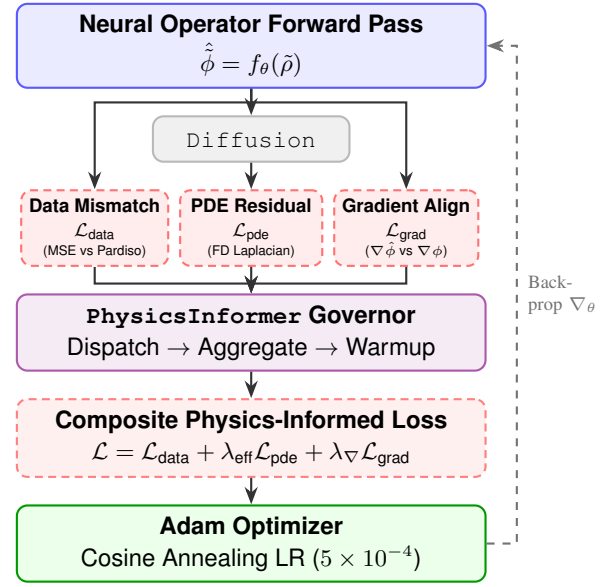


Fig. 4. Physics-informed training pipeline. The forward pass produces $\hat{\phi}$, from which three loss terms are computed: data MSE, FD-based PDE residual (via the Diffusion stencil engine), and gradient alignment. The PhysicsInformer aggregates these with epoch-dependent warmup weighting into the composite loss, which drives Adam optimisation with cosine LR annealing.

B. Why FD Stencils Instead of Autograd

In many PINN settings, the PDE residual is computed with automatic differentiation (AD). This is effective when the model takes one coordinate (x, y) and returns one scalar value. Our setting is different: the model predicts a full 256×256 field in one pass through stacked FFT/iFFT layers.

If AD is used to compute second derivatives at all 65,536 grid points, PyTorch must retain a large derivative graph for the spectral operations. In early experiments, this graph exhausted GPU VRAM and made each training epoch about $50 \times$ slower than a finite-difference (FD) approach.

We therefore compute spatial derivatives *after* the forward pass using a detached 5-point FD stencil. This avoids graph-retention overhead, keeps memory use stable, and is simple to parallelize on tensors. It also matches the stencil used in the C++ PIC solver, so training and deployment use the same numerical operator. For this second-order centred stencil, the leading discretisation error scales with h^2 ; with $h = 1/255$, $h^2 \approx 1.5 \times 10^{-5}$. This truncation error is negligible compared with training residuals of $\mathcal{O}(10^{-1})$.

C. Physics Loss Backend

Physics supervision is handled by two modules with separate roles. The Diffusion class computes the 5-point Laplacian on the predicted PyTorch field using shifted-tensor operations, producing a spatial residual map without retaining an AD graph. The PhysicsInformer class then post-processes this map: it removes a 2-cell border on each edge, forms the normalized physics loss $\mathcal{L}_{\text{pde}}$, and applies the linear warmup schedule for $\lambda_{\text{eff}}(e)$. During early epochs, λ_{eff} is kept near zero so the data

TABLE IV
TRAINING HYPERPARAMETERS.

Hyperparameter	Value
Physics loss weight λ_{phys}	0.1
Gradient loss weight λ_{∇}	0.05
Warmup duration E_{warm}	5 epochs
Training epochs	180 (from scratch)
Best checkpoint	Epoch 177
Initial learning rate η_0	1.0×10^{-4}
Minimum learning rate η_{min}	1.0×10^{-6}
Gradient clip norm	1.0
Optimiser	Adam
LR schedule	Cosine annealing
Batch size	1
PDE grid spacing h	$1/255 \approx 0.004$
Border crop (each side)	2 cells

term can first build a stable global field before stronger PDE enforcement is introduced.

D. Physics Warmup Schedule

Batch size is fixed at 1 because four FFT-based spectral layers on a 256×256 grid saturate available GPU VRAM. A warmup schedule is therefore essential for stable optimization. At epoch 0, predictions are still close to random. When the 5-point FD Laplacian is applied to such fields, the resulting gradient scale is $\mathcal{O}(h^{-2}) \approx 62,500$, about $625 \times$ larger than the data-loss gradient. If the physics term is fully active from the first epoch, it dominates the update direction and prevents the network from first learning the correct global field shape.

The warmup schedule ramps the physics weight linearly:

$$\lambda_{\text{eff}}(e) = \lambda_{\text{phys}} \min\left(1, \frac{e+1}{E_{\text{warm}}}\right), \quad (16)$$

with $\lambda_{\text{phys}} = 0.1$ and $E_{\text{warm}} = 5$. This means early epochs are driven mainly by supervised data fitting, which builds a coherent potential field. The PDE term then ramps in gradually and reaches full strength only after that coarse structure is stable, improving physical consistency without destabilizing training.

E. Optimiser and Schedule

We use Adam with gradient-norm clipping at $\|\nabla_{\theta}\|_2 \leq 1.0$. The learning rate follows cosine annealing:

$$\eta_t = \eta_{\text{min}} + \frac{1}{2}(\eta_0 - \eta_{\text{min}}) \left(1 + \cos \frac{\pi t}{T_{\text{max}}}\right), \quad (17)$$

with $\eta_0 = 1.0 \times 10^{-4}$ and $\eta_{\text{min}} = 1.0 \times 10^{-6}$. Training was performed from scratch for 180 epochs. Best validation performance was recorded at epoch 177.

VI. INFERENCE PIPELINE

Once trained, all weights are frozen. Each new charge density $\rho^{(n)}$ passes through a deterministic inference pipeline (Fig. 5):

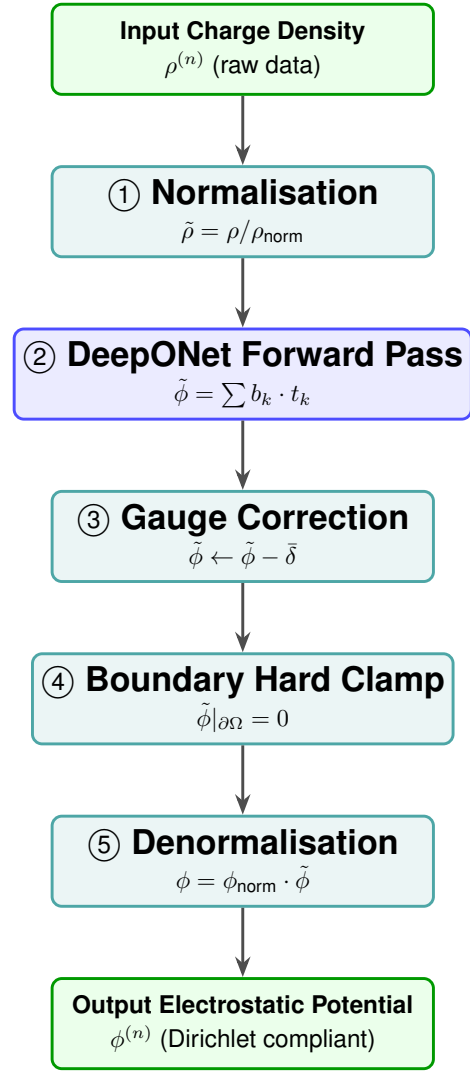


Fig. 5. Inference pipeline for a single charge density input. The post-processing chain (normalise $\rightarrow$ forward pass $\rightarrow$ gauge fix $\rightarrow$ hard clamp $\rightarrow$ denormalise) guarantees exact Dirichlet compliance and correct physical units in the output potential field.

- 1) **Normalisation:** Scale the raw input to the training distribution:

$$\tilde{\rho}^{(n)} = \frac{\rho^{(n)}}{\rho_{\text{norm}}}$$

- 2) **Forward Pass:** Evaluate the pre-trained operator via the branch-trunk decomposition:

$$\hat{\phi}^{(n)} = \sum_{k=1}^4 b_k(\tilde{\rho}^{(n)}) \cdot t_k(x, y)$$

- 3) **Gauge Correction:** Subtract the mean boundary prediction to uniquely select the grounded Dirichlet gauge:

$$\delta^{(n)} = \frac{1}{4}(\mu_{\text{top}} + \mu_{\text{bot}} + \mu_{\text{left}} + \mu_{\text{right}}) \quad (18)$$

$$\hat{\phi}^{(n)} \leftarrow \hat{\phi}^{(n)} - \delta^{(n)}$$

Because $\delta^{(n)}$ is spatially constant, $\nabla^2 \delta^{(n)} = 0$; the correction shifts the entire field without altering interior physics.

- 4) **Hard Clamp:** Explicitly zero the boundary edges to enforce the Dirichlet condition:

$$\hat{\phi}^{(n)}|_{\partial\Omega} = 0$$

- 5) **Denormalisation:** Restore the prediction to physical voltage units:

$$\hat{\phi}^{(n)} = \phi_{\text{norm}} \cdot \hat{\phi}^{(n)}$$

- 6) **Metrics Evaluation:** Compute validation accuracy against the ground truth reference $\phi^{(n)}$ using MAE, RMSE, APE, and SSIM.

A. Error Metrics

Let $e_{i,j}^{(n)} = \hat{\phi}_{i,j}^{(n)} - \phi_{i,j}^{(n)}$ denote the per-cell error. The following metrics characterize complementary aspects of prediction quality:

- **Mean Error (ME):** Signed bias, which reveals whether the model systematically over- or under-predicts.

$$\text{ME} = \frac{1}{HW} \sum_{i,j} e_{i,j}$$

- **Mean Absolute Error (MAE):** Primary pointwise accuracy indicator; not inflated by boundary zeros.

$$\text{MAE} = \frac{1}{HW} \sum_{i,j} |e_{i,j}|$$

- **Root Mean Squared Error (RMSE):** Penalises large localised deviations more heavily than MAE, revealing worst-case outlier cells.

$$\text{RMSE} = \sqrt{\frac{1}{HW} \sum_{i,j} e_{i,j}^2}$$

- **Absolute Percentage Error (APE):** Relative error, regularised by a floor ϵ_b to avoid division by zero at grounded walls.

$$\text{APE} = \frac{100}{HW} \sum_{i,j} \frac{|e_{i,j}|}{\max(|\phi_{i,j}|, \epsilon_b)}$$

- **Structural Similarity Index (SSIM):**

$$\text{SSIM}(u, v) = \frac{(2\mu_u\mu_v + c_1)(2\sigma_{uv} + c_2)}{(\mu_u^2 + \mu_v^2 + c_1)(\sigma_u^2 + \sigma_v^2 + c_2)}, \quad (19)$$

where μ , σ^2 , σ_{uv} denote local mean, variance, and cross-covariance computed with an 11×11 Gaussian window. SSIM captures spatial structural coherence—distortions that leave the pointwise MSE unchanged but corrupt the field's gradient map will show up here.

Caveat on APE. Near grounded boundaries where $\phi_{\text{true}} \rightarrow 0$, even negligible absolute errors of order millivolts can produce inflated percentage errors. MAE and RMSE are therefore the primary accuracy indicators throughout this work; APE is reported as a secondary reference.

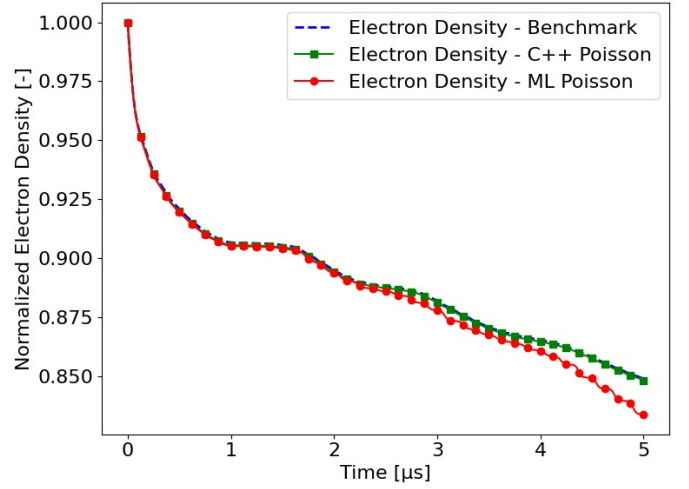


Fig. 6. Normalized electron density over 2000 PIC iterations. The ML Poisson solver (red) is compared with the serial C++ Poisson solver (green, square markers) and the expected benchmark trajectory (blue, dashed).

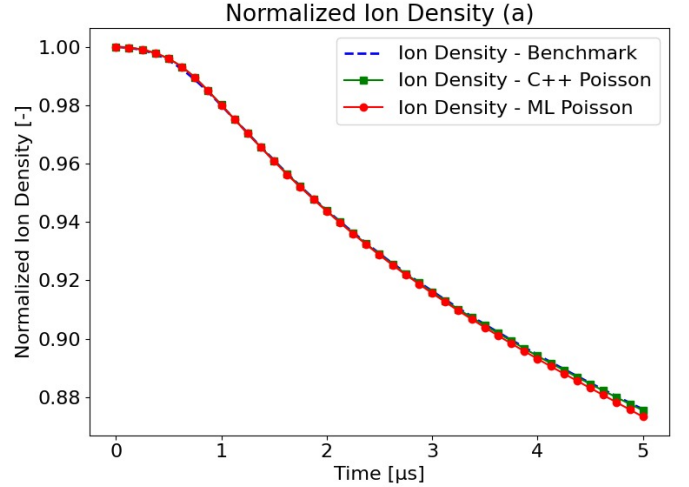


Fig. 7. Normalized ion density over 2000 PIC iterations. The ML Poisson solver (red) follows the expected ion-density decay and remains close to the serial C++ Poisson solver (green, square markers) and benchmark trajectory (blue, dashed).

VII. RESULTS AND DISCUSSION

A. Dynamic Validation

For a PIC surrogate, the key test is not only snapshot accuracy on held-out data. The more important question is whether the model can run inside a live plasma simulation for many time-steps without introducing non-physical energy growth or numerical drift. To test this, the trained neural Poisson solver was inserted into the PIC loop and compared against the serial C++ Poisson solver and the expected benchmark trajectory over 2000 PIC iterations. Fig. 6 and Fig. 7 show that the learned field solver tracks the expected electron and ion density decay over the full transient.

B. Training Convergence

Fig. 8 shows four validation metrics over 180 epochs. All four improve consistently. MSE decreases from 0.2482 at epoch 0 to 4.862×10^{-4} at the best checkpoint (epoch 177). SSIM rises from 0.868 to 0.993, indicating strong structural agreement, while APE falls from 70.33% to 2.994%. The normalized PDE residual also drops from 0.521 to 0.263, indicating improved physics consistency during training.

The training loss remains above the validation loss in Fig. 8. In this setting, that separation is expected and does not by itself indicate overfitting. The training objective includes the additional physics terms $\mathcal{L}_{\text{pde}}$ and $\mathcal{L}_{\text{grad}}$, whereas the validation curve reports data MSE only; the two curves therefore measure different objectives.

C. Loss Components

Table V summarizes the active composite objective in the current training run. The data term controls pointwise potential accuracy, the normalized PDE residual enforces Poisson consistency, and the gradient-alignment term improves electric-field behavior for PIC updates. Hard Dirichlet clamping is applied as deterministic post-processing rather than as a learned penalty, ensuring exact zero-potential boundaries.

TABLE V
COMPOSITE LOSS COMPONENTS IN THE CURRENT TRAINING RUN.

Component	Weight	Epoch 177 unweighted value
$\mathcal{L}_{\text{data}}$	1.0	4.743×10^{-4}
$\mathcal{L}_{\text{pde}}$	0.1	0.264
$\mathcal{L}_{\text{grad}}$	0.05	0.568

Weights are from the current configuration; component values are epoch 177 training-log metrics.

D. Full Validation Summary

Across all 4002 held-out samples (Table VI), the model achieves average RMSE = 4.217788×10^{-2} V, average APE 3.5599%, and median APE 1.5001%. The lower median relative to the mean reflects that most validation samples have small percentage error, while a smaller number of near-boundary or high-gradient cases raise the average APE.

The normalized PDE residual converges to 0.264 rather than zero. In practice, this value represents the stable balance reached among data fitting, PDE enforcement, and gradient-alignment constraints.

E. Spatial Error Distribution

Fig. 9 shows the per-cell error statistics across all 4002 validation fields. The signed-error map shows a mild uneven pattern, indicating that the model does not achieve identical accuracy in all directions. This may result from the limited representation of fine-scale details in the FNO branch. The MAE map is highest in the central region, where charge variation and field gradients are strongest. Errors remain close to zero near the grounded boundaries because the boundary condition is enforced exactly.

TABLE VI
FULL EVALUATION ON 4002 UNSEEN VALIDATION SAMPLES.

Metric	Value
Average RMSE	4.217788×10^{-2} V
Average APE	3.5599%
Median APE	1.5001%

TABLE VII
WORST-CASE VALIDATION SAMPLES.

Rank	Sample	MAE (V)
1	0	9.906×10^{-2}
2	3849	9.866×10^{-2}
3	3852	9.756×10^{-2}
4	3851	9.755×10^{-2}

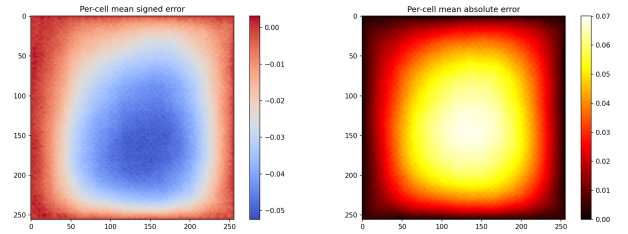


Fig. 9. Per-cell error statistics over 4002 validation samples (256×256). **Left:** Mean signed error showing a bias dipole. **Right:** Mean absolute error concentrated in mid-domain where charge variance is maximal.

F. Qualitative Predictions

Fig. 10 compares the input charge density, ground-truth potential, predicted potential, and absolute error for four representative validation samples. In all cases, the predicted potential closely matches the reference distribution and preserves the overall spatial structure of the field. The model reproduces smooth interior variation and sharp boundary-adjacent changes with good accuracy. Absolute error remains low over most of the domain, with larger values confined to a few localized regions. The visual comparison is consistent with the aggregate validation metrics.

G. Bottleneck Analysis

Table VII lists the four hardest validation samples. The largest per-sample MAE values are for samples 0 and 3849: 9.906×10^{-2} V and 9.866×10^{-2} V. These cases include separated charge clusters with stronger high-frequency content above $k = 24$. This behavior reflects a model-capacity trade-off rather than a training breakdown. The current configuration uses $k_{\text{max}} = 24$ and achieves 3.5599% average APE over the full validation set. The remaining worst-case error comes from frequencies intentionally excluded to fit available GPU VRAM.

H. Spectral Mode Selection

The number of FNO modes k_{max} determines the highest spatial frequency the model can represent. On a 256×256 grid

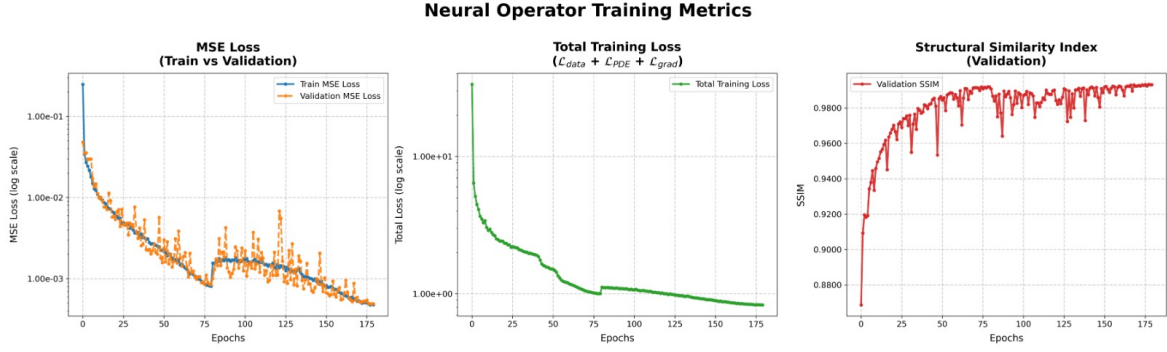


Fig. 8. Training and validation MSE loss (log scale) showing model generalisation across epochs, alongside the total composite training loss ($\mathcal{L}_{data} + \mathcal{L}_{PDE} + \mathcal{L}_{grad}$) reflecting joint data and physics constraint satisfaction, and validation SSIM tracking the structural fidelity of predicted Poisson solutions. All three metrics collectively demonstrate stable convergence of the neural operator throughout training.

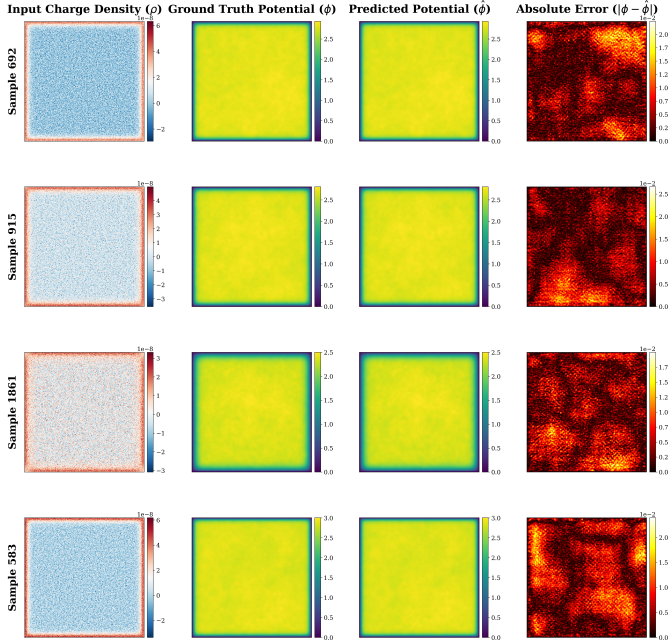


Fig. 10. Qualitative prediction results for four representative validation samples. Each row shows (left to right): input charge density ρ , DeepONet-predicted potential $\hat{\phi}$, Pardiso reference potential ϕ , and absolute error $|\hat{\phi} - \phi|$. Samples span diverse charge topologies, consistent with the full-validation median APE of 1.5001%.

TABLE VIII
SPECTRAL COVERAGE IN THE CURRENT CONFIGURATION.

$k_{\max}$	f_{spec}	Resolved features
24	18.8%	Sharp sheath boundaries (this work)

the Nyquist limit is $k_{\text{Nyq}} = N/2 = 128$; the fraction of the spectrum captured is $f_{\text{spec}} = k_{\max}/128$. Table VIII summarises the current spectral setting.

At $k_{\max} = 24$, the model resolves spatial structures as narrow as $256/24 \approx 11$ grid cells—sufficient to capture plasma sheaths, which are the dominant high-gradient features in

these simulations. Increasing beyond 24 requires storing larger complex weight tensors per spectral layer; memory cost grows as $\mathcal{O}(k_{\max}^2)$.

VIII. CONCLUSION

We demonstrated that a branch–trunk DeepONet—with an FNO branch encoding global charge structure in the frequency domain and a coordinate MLP trunk evaluating the solution at continuous spatial points—can replace the Poisson field solve inside a PIC simulation loop without destabilising plasma dynamics. The surrogate achieves an average RMSE of 4.217788×10^{-2} V, average APE of 3.5599%, and median APE of 1.5001% on 4002 unseen charge configurations, validated across seven distinct plasma scenarios.

Two engineering decisions proved critical in practice. Replacing automatic differentiation with explicit finite-difference stencils for the physics loss eliminated GPU out-of-memory failures that arose from retaining AD graphs through four stacked FFT layers. Hard boundary clamping as a deterministic post-processing stage—rather than a soft penalty term—guaranteed exact Dirichlet compliance on every forward pass without consuming network capacity.

The residual limitations are concrete and addressable. The ≈ 0.264 normalised PDE residual at convergence reflects a trade-off among competing loss terms, not a fundamental accuracy ceiling. Extending the spectral mode budget beyond $k_{\max} = 24$ may reduce the worst-case errors in high-frequency charge configurations at the cost of increased VRAM. Both directions offer clear paths for future improvement.

REFERENCES

- [1] C. K. Birdsall and A. B. Langdon, *Plasma Physics via Computer Simulation*, CRC Press, 1991.
- [2] T. Chen and H. Chen, “Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems,” *IEEE Trans. Neural Networks*, vol. 6, no. 4, pp. 911–917, 1995.
- [3] M. Desai, P. Ghosh, A. Kumar, and B. Chaudhury, “Deep-Learning Architecture-Based Approach for 2-D-Simulation of Microwave Plasma Interaction,” *IEEE Trans. Microw. Theory Techn.*, vol. 70, no. 12, pp. 5359–5367, Dec. 2022.

- [4] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.
- [5] L. Lu, P. Jin, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet," *Nature Machine Intelligence*, vol. 3, pp. 218–229, 2021.
- [6] Z. Li *et al.*, "Fourier neural operator for parametric PDEs," in *Proc. 9th Int. Conf. Learning Representations (ICLR)*, 2021.
- [7] L. C. Evans, *Partial Differential Equations*, 2nd ed. AMS, 2010.